

Rajshahi University of Engineering & Technology (RUET)

Department of Electrical and Computer Engineering

Database Systems Sessional

Course Code: ECE 2216

Lab Report No. 03

SQL: Aliasing, Sorting, Filtering, Grouping, Set Operations & Subqueries

Submitted To,

Mst. Nafia Islam Shishir
Lecturer
Dept. of ECE, RUET

Submitted By,

Md. Yousuf
Roll: 2310023
Series: 2023
Dept. of ECE, RUET

Contents

1	Theory	3
2	Table Creation and Data Insertion	4
2.1	Table 1: Student	4
2.2	Table 2: Alumni	5
3	Lab Tasks	6
3.1	Task 1: Display Student Names in Uppercase and Lowercase	6
3.2	Task 2: Rename the Column Student_Name as Name Using AS	7
3.3	Task 3: Display All Students Ordered by Department (Ascending) and CGPA (Descending)	7
3.4	Task 4: Display Students Whose CGPA is BETWEEN 3.50 AND 4.00	8
3.5	Task 5: Display Students Whose Department is IN ('CSE','EEE')	9
3.6	Task 6: Display Students Whose Department is NOT IN ('BBA')	9
3.7	Task 7: Display the Average CGPA of Each Department Using GROUP BY . .	10
3.8	Task 8: Display Departments Having Average CGPA Greater Than 3.50 Using HAVING	11
3.9	Task 9: Display All Unique Student Names from Student and Alumni Using UNION	11
3.10	Task 10: Display Students Whose CGPA is Higher Than the Average CGPA Using a Nested Subquery	12
4	Discussion	14

List of Figures

3.1	Output of Task 1		6
3.2	Output of Task 2		7
3.3	Output of Task 3		8
3.4	Output of Task 4		8
3.5	Output of Task 5		9
3.6	Output of Task 6		10
3.7	Output of Task 7		10
3.8	Output of Task 8		11
3.9	Output of Task 9		12
3.10	Output of Task 10		13

Chapter 1

Theory

Structured Query Language (SQL) is the standard language used to create, manage, and query relational databases. A relational database organizes data into tables (relations), where each table consists of rows (records) and columns (attributes). SQL commands are broadly divided into categories such as Data Definition Language (DDL), used to define database structures (e.g. `CREATE TABLE`), and Data Manipulation Language (DML), used to insert, update, retrieve, and delete data (e.g. `INSERT`, `SELECT`, `UPDATE`, `DELETE`).

In this lab, the `SELECT` statement is used extensively along with several of its clauses and operators:

- **Aliasing (AS):** Used to give a column or table a temporary, more readable name for the duration of a query.
- **Sorting (ORDER BY):** Used to arrange the result set in ascending (`ASC`) or descending (`DESC`) order based on one or more columns.
- **Filtering (WHERE, BETWEEN, IN, NOT IN):** Used to restrict the rows returned by a query based on specified conditions.
- **Grouping (GROUP BY, HAVING):** `GROUP BY` groups rows that share a common value into summary rows, and `HAVING` filters those groups based on aggregate conditions, unlike `WHERE`, which filters individual rows before grouping.
- **Set Operations (UNION):** Used to combine the result sets of two or more `SELECT` statements, automatically removing duplicate rows.
- **Subqueries:** A query nested inside another query, often used to compare a value against an aggregate result, such as an average.

These tools allow flexible and powerful retrieval, transformation, and summarization of data stored in relational tables, forming the foundation of practical database querying.

Chapter 2

Table Creation and Data Insertion

Two tables were created for this lab: `Student`, which holds academic records of currently enrolled students, and `Alumni`, which holds records of former students. Both tables share a similar structure so that they can later be combined using set operations.

2.1 Table 1: Student

```
1 CREATE TABLE Student (  
2     Student_ID    VARCHAR(10) PRIMARY KEY,  
3     Student_Name  VARCHAR(50),  
4     Gender        VARCHAR(10),  
5     Department    VARCHAR(10),  
6     Semester      VARCHAR(5),  
7     CGPA          DECIMAL(3,2),  
8     Age           INT,  
9     Email         VARCHAR(50)  
10 );
```

Listing 2.1: Creating the Student table

```
1 INSERT INTO Student VALUES  
2 ('2310023', 'Yousuf', 'Male', 'EEE', '7th', 3.85, 22, 'yousuf@ruet.  
   edu'),  
3 ('2310024', 'Mahi', 'Male', 'CSE', '5th', 3.42, 21, 'mahi@ruet.edu  
   '),  
4 ('2310025', 'Mohon', 'Male', 'EEE', '6th', 3.92, 22, 'mohon@ruet.  
   edu'),  
5 ('2310026', 'Sakib', 'Male', 'BBA', '8th', 3.25, 23, 'sakib@ruet.  
   edu'),  
6 ('2310027', 'Moshi', 'Male', 'CSE', '7th', 3.70, 21, 'moshi@ruet.  
   edu'),  
7 ('2310028', 'Rafi', 'Male', 'EEE', '6th', 3.10, 22, 'rafi@ruet.edu  
   '),  
8 ('2310029', 'Nabil', 'Male', 'BBA', '8th', 3.95, 23, NULL),  
9 ('2310030', 'Tanvir', 'Male', 'CSE', '5th', 2.95, 20, 'tanvir@ruet.  
   edu');
```

Listing 2.2: Inserting records into the Student table

Table 2.1 shows the resulting data stored in the `Student` table after the insertion.

Table 2.1: Student Table Data

Student_ID	Student_Name	Gender	Department	Semester	CGPA	Age	Email
2310023	Yousuf	Male	EEE	7th	3.85	22	yousuf@ruet.edu
2310024	Mahi	Male	CSE	5th	3.42	21	mahi@ruet.edu
2310025	Mohon	Male	EEE	6th	3.92	22	mohon@ruet.edu
2310026	Sakib	Male	BBA	8th	3.25	23	sakib@ruet.edu
2310027	Moshi	Male	CSE	7th	3.70	21	moshi@ruet.edu
2310028	Rafi	Male	EEE	6th	3.10	22	rafi@ruet.edu
2310029	Nabil	Male	BBA	8th	3.95	23	—
2310030	Tanvir	Male	CSE	5th	2.95	20	tanvir@ruet.edu

2.2 Table 2: Alumni

```

1 CREATE TABLE Alumni (
2     Student_ID    VARCHAR(10) PRIMARY KEY,
3     Student_Name  VARCHAR(50),
4     Department    VARCHAR(10)
5 );

```

Listing 2.3: Creating the Alumni table

```

1 INSERT INTO Alumni VALUES
2 ('AL001', 'Arif', 'CSE'),
3 ('AL002', 'Mim', 'BBA'),
4 ('AL003', 'Fahim', 'EEE'),
5 ('2310027', 'Moshi', 'CSE'),
6 ('2310025', 'Mohon', 'EEE');

```

Listing 2.4: Inserting records into the Alumni table

Table 2.2 shows the resulting data stored in the `Alumni` table. Notice that the last two records share the same `Student_ID` and `Department` as two rows already present in the `Student` table (Moshi and Mohon). This overlap was introduced intentionally so that the `UNION` operation performed later in the lab can be observed removing duplicate names.

Table 2.2: Alumni Table Data

Student_ID	Student_Name	Department
AL001	Arif	CSE
AL002	Mim	BBA
AL003	Fahim	EEE
2310027	Moshi	CSE
2310025	Mohon	EEE

Chapter 3

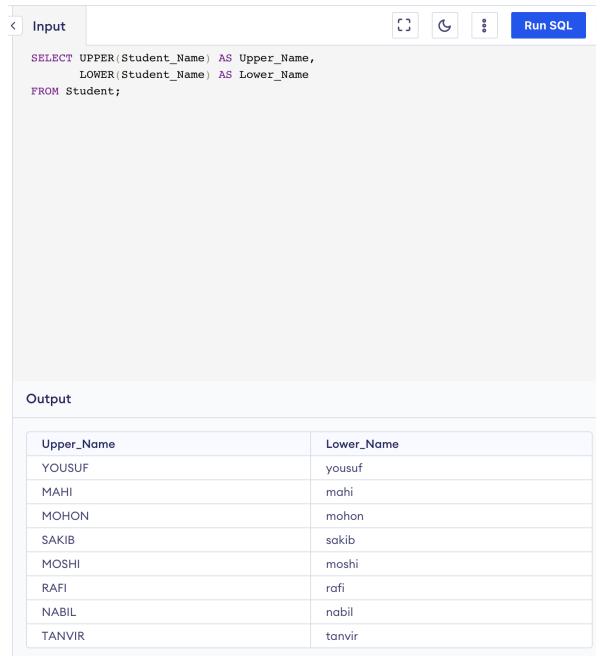
Lab Tasks

3.1 Task 1: Display Student Names in Uppercase and Lowercase

The UPPER () and LOWER () string functions were used to convert the Student_Name column to fully uppercase and fully lowercase text respectively, within the same query.

```
1 SELECT UPPER(Student_Name) AS Upper_Name,  
2     LOWER(Student_Name) AS Lower_Name  
3 FROM Student;
```

Listing 3.1: Query for Task 1



The screenshot shows a SQL query editor with an 'Input' tab. The query is: `SELECT UPPER(Student_Name) AS Upper_Name, LOWER(Student_Name) AS Lower_Name FROM Student;`. Below the query is an 'Output' tab displaying a table with two columns: 'Upper_Name' and 'Lower_Name'. The table contains eight rows of student names in both uppercase and lowercase formats.

Upper_Name	Lower_Name
YOUSUF	yousuf
MAHI	mahi
MOHON	mohon
SAKIB	sakib
MOSHI	moshi
RAFI	rafi
NABIL	nabil
TANVIR	tanvir

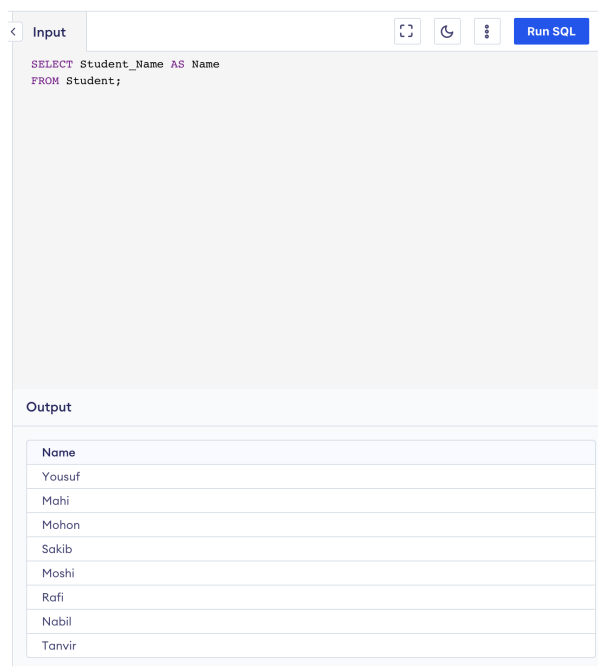
Figure 3.1: Output of Task 1

3.2 Task 2: Rename the Column Student_Name as Name Using AS

The AS keyword was used to create a temporary alias, displaying the Student_Name column under the heading Name in the result set, without altering the actual column name in the table.

```
1 SELECT Student_Name AS Name
2 FROM Student;
```

Listing 3.2: Query for Task 2



The screenshot shows a SQL query editor with an 'Input' tab and a 'Run SQL' button. The query is: `SELECT Student_Name AS Name FROM Student;`. Below the input, the 'Output' tab displays a table with the results of the query.

Name
Yousuf
Mahi
Mohon
Sakib
Moshi
Rafi
Nabil
Tanvir

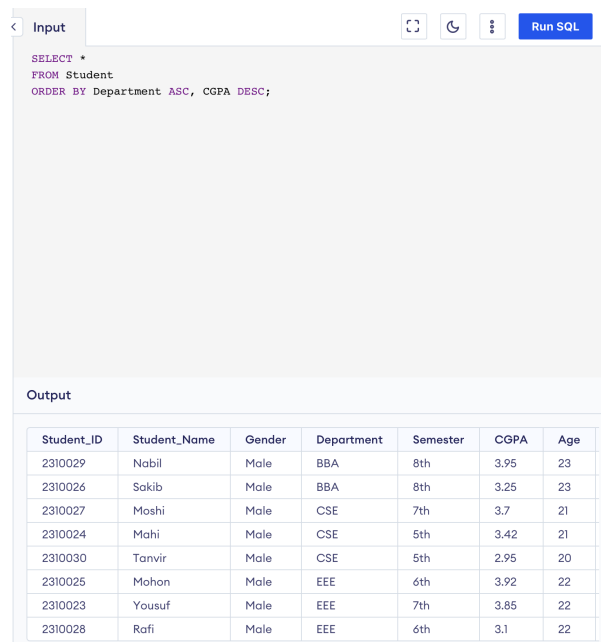
Figure 3.2: Output of Task 2

3.3 Task 3: Display All Students Ordered by Department (Ascending) and CGPA (Descending)

The ORDER BY clause was used with multiple columns. Rows are first sorted alphabetically by Department in ascending order, and within each department, rows are sorted by CGPA in descending order.

```
1 SELECT *
2 FROM Student
3 ORDER BY Department ASC, CGPA DESC;
```

Listing 3.3: Query for Task 3



The screenshot shows a SQL query editor with an 'Input' tab. The query is: `SELECT * FROM Student ORDER BY Department ASC, CGPA DESC;`. Below the input, there is an 'Output' section displaying a table with 7 columns: Student_ID, Student_Name, Gender, Department, Semester, CGPA, and Age. The table contains 9 rows of student data, sorted by Department and then by CGPA in descending order.

Student_ID	Student_Name	Gender	Department	Semester	CGPA	Age
2310029	Nabil	Male	BBA	8th	3.95	23
2310026	Sakib	Male	BBA	8th	3.25	23
2310027	Moshi	Male	CSE	7th	3.7	21
2310024	Mahi	Male	CSE	5th	3.42	21
2310030	Tanvir	Male	CSE	5th	2.95	20
2310025	Mohon	Male	EEE	6th	3.92	22
2310023	Yousuf	Male	EEE	7th	3.85	22
2310028	Rafi	Male	EEE	6th	3.1	22

Figure 3.3: Output of Task 3

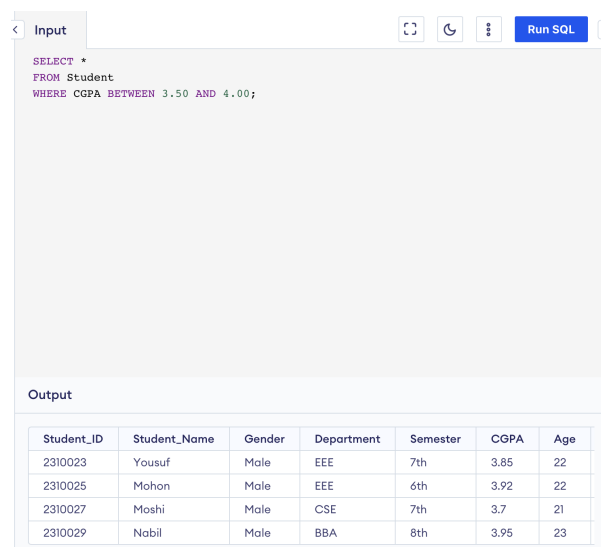
3.4 Task 4: Display Students Whose CGPA is BETWEEN 3.50 AND 4.00

The BETWEEN operator was used to filter rows whose CGPA value lies within the inclusive range of 3.50 to 4.00.

```

1 SELECT *
2 FROM Student
3 WHERE CGPA BETWEEN 3.50 AND 4.00;
```

Listing 3.4: Query for Task 4



The screenshot shows a SQL query editor with an 'Input' tab. The query is: `SELECT * FROM Student WHERE CGPA BETWEEN 3.50 AND 4.00;`. Below the input, there is an 'Output' section displaying a table with 7 columns: Student_ID, Student_Name, Gender, Department, Semester, CGPA, and Age. The table contains 4 rows of student data, filtered by CGPA between 3.50 and 4.00.

Student_ID	Student_Name	Gender	Department	Semester	CGPA	Age
2310023	Yousuf	Male	EEE	7th	3.85	22
2310025	Mohon	Male	EEE	6th	3.92	22
2310027	Moshi	Male	CSE	7th	3.7	21
2310029	Nabil	Male	BBA	8th	3.95	23

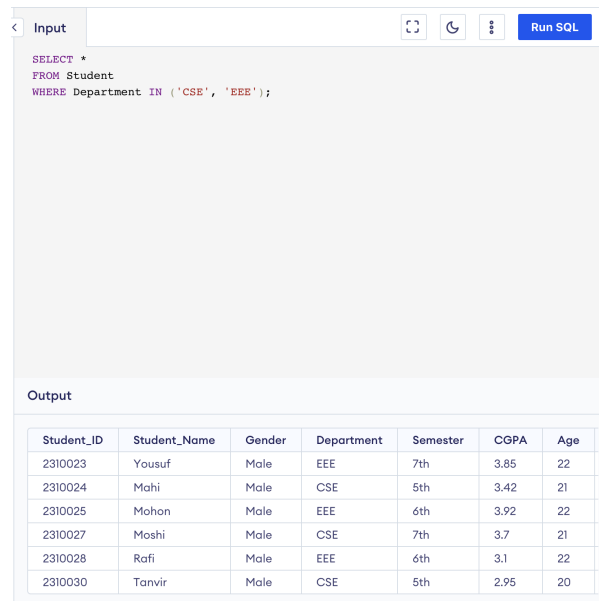
Figure 3.4: Output of Task 4

3.5 Task 5: Display Students Whose Department is IN ('CSE','EEE')

The IN operator was used to match rows where Department is either CSE or EEE, avoiding the need for multiple OR conditions.

```
1 SELECT *
2 FROM Student
3 WHERE Department IN ('CSE', 'EEE');
```

Listing 3.5: Query for Task 5



The screenshot shows a SQL query editor with an 'Input' tab. The query is: `SELECT * FROM Student WHERE Department IN ('CSE', 'EEE');`. Below the query is an 'Output' tab displaying a table with 7 columns: Student_ID, Student_Name, Gender, Department, Semester, CGPA, and Age. The table contains 7 rows of student data.

Student_ID	Student_Name	Gender	Department	Semester	CGPA	Age
2310023	Yousuf	Male	EEE	7th	3.85	22
2310024	Mahi	Male	CSE	5th	3.42	21
2310025	Mohon	Male	EEE	6th	3.92	22
2310027	Moshi	Male	CSE	7th	3.7	21
2310028	Rafi	Male	EEE	6th	3.1	22
2310030	Tanvir	Male	CSE	5th	2.95	20

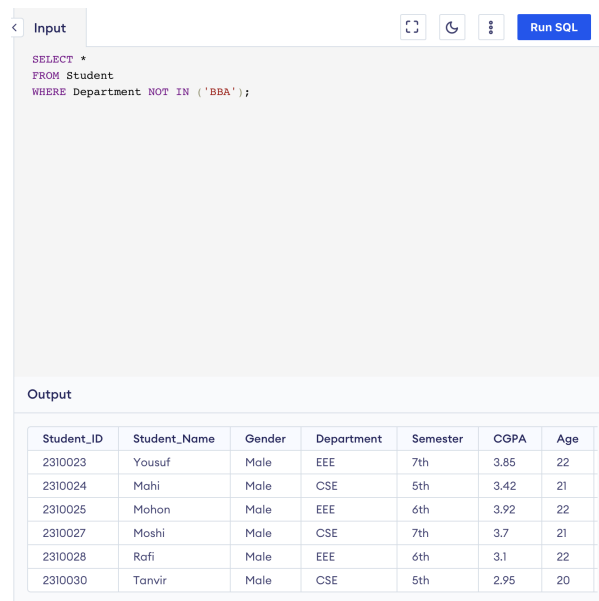
Figure 3.5: Output of Task 5

3.6 Task 6: Display Students Whose Department is NOT IN ('BBA')

The NOT IN operator was used to exclude rows where Department equals BBA, returning only students from all other departments.

```
1 SELECT *
2 FROM Student
3 WHERE Department NOT IN ('BBA');
```

Listing 3.6: Query for Task 6



Input

```
SELECT *
FROM Student
WHERE Department NOT IN ('BBA');
```

Run SQL

Output

Student_ID	Student_Name	Gender	Department	Semester	CGPA	Age
2310023	Yousuf	Male	EEE	7th	3.85	22
2310024	Mahi	Male	CSE	5th	3.42	21
2310025	Mohon	Male	EEE	6th	3.92	22
2310027	Moshi	Male	CSE	7th	3.7	21
2310028	Rafi	Male	EEE	6th	3.1	22
2310030	Tanvir	Male	CSE	5th	2.95	20

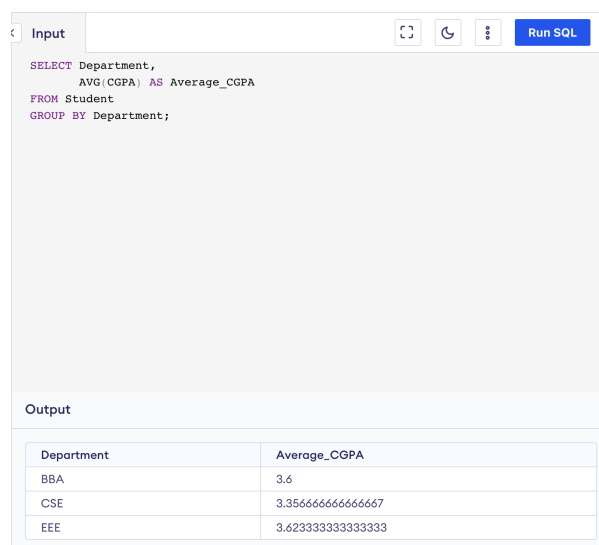
Figure 3.6: Output of Task 6

3.7 Task 7: Display the Average CGPA of Each Department Using GROUP BY

The GROUP BY clause was used to group rows by Department, and the AVG () aggregate function was applied to compute the mean CGPA within each group.

```
1 SELECT Department,
2     AVG(CGPA) AS Average_CGPA
3 FROM Student
4 GROUP BY Department;
```

Listing 3.7: Query for Task 7



Input

```
SELECT Department,
       AVG(CGPA) AS Average_CGPA
FROM Student
GROUP BY Department;
```

Run SQL

Output

Department	Average_CGPA
BBA	3.6
CSE	3.35666666666667
EEE	3.62333333333333

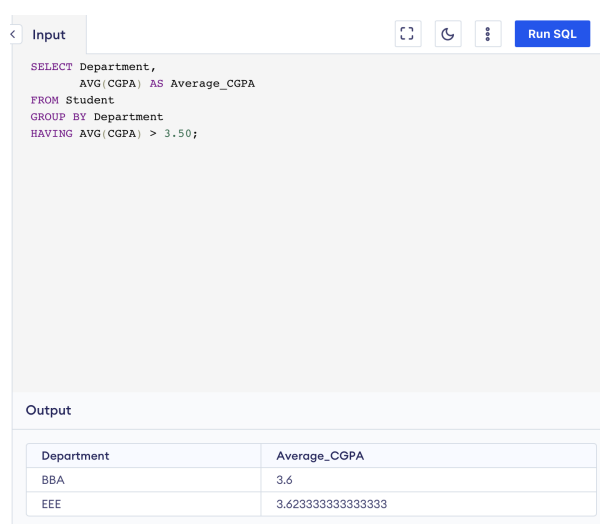
Figure 3.7: Output of Task 7

3.8 Task 8: Display Departments Having Average CGPA Greater Than 3.50 Using HAVING

After grouping rows by Department and computing the average CGPA, the HAVING clause was used to filter out groups whose average CGPA is not greater than 3.50. Unlike WHERE, HAVING operates on the aggregated group results.

```
1 SELECT Department,  
2     AVG(CGPA) AS Average_CGPA  
3 FROM Student  
4 GROUP BY Department  
5 HAVING AVG(CGPA) > 3.50;
```

Listing 3.8: Query for Task 8



The screenshot shows a SQL query editor with an 'Input' tab and a 'Run SQL' button. The query is the same as in Listing 3.8. Below the query, the 'Output' tab displays a table with two columns: 'Department' and 'Average_CGPA'. The table contains two rows: 'BBA' with an average CGPA of 3.6, and 'EEE' with an average CGPA of 3.6233333333333333.

Department	Average_CGPA
BBA	3.6
EEE	3.6233333333333333

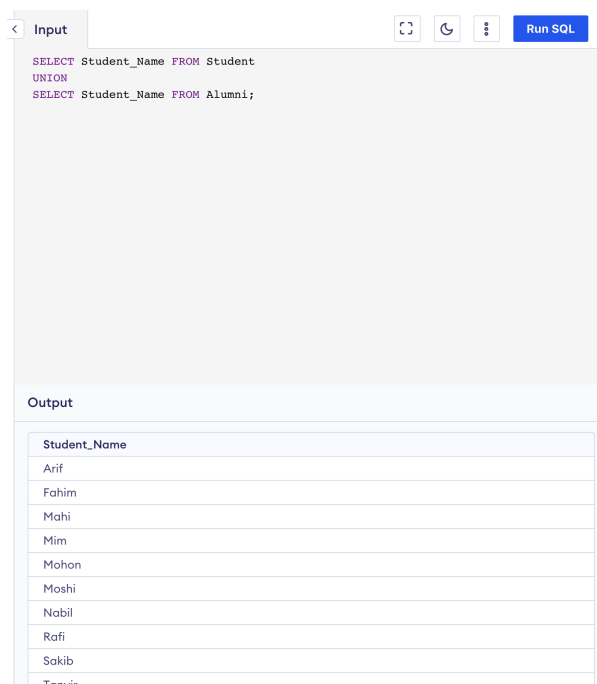
Figure 3.8: Output of Task 8

3.9 Task 9: Display All Unique Student Names from Student and Alumni Using UNION

The UNION operator was used to combine the Student_Name column from both the Student and Alumni tables into a single result set, automatically eliminating duplicate names such as “Moshi” and “Mohon,” which appear in both tables.

```
1 SELECT Student_Name FROM Student  
2 UNION  
3 SELECT Student_Name FROM Alumni;
```

Listing 3.9: Query for Task 9



The screenshot shows a SQL query editor with an 'Input' tab. The query is a UNION of two SELECT statements. The output is displayed in a table with the header 'Student_Name' and ten rows of student names.

```
SELECT Student_Name FROM Student
UNION
SELECT Student_Name FROM Alumni;
```

Student_Name
Arif
Fahim
Mahi
Mim
Mohon
Moshi
Nabil
Rafi
Sakib
Tasvir

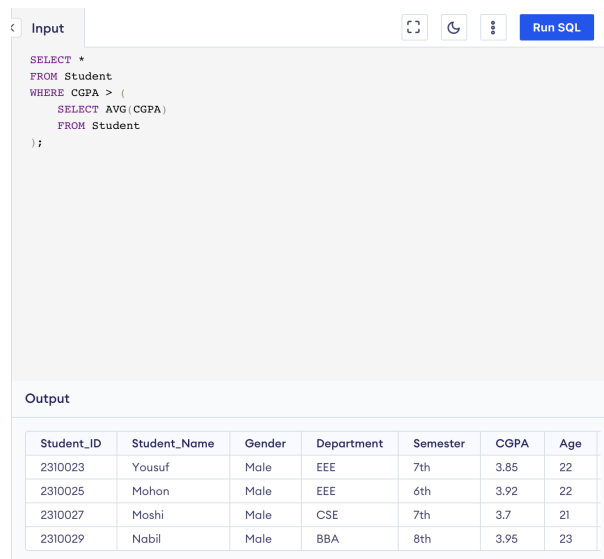
Figure 3.9: Output of Task 9

3.10 Task 10: Display Students Whose CGPA is Higher Than the Average CGPA Using a Nested Subquery

A nested subquery was used inside the WHERE clause. The inner query first calculates the overall average CGPA of all students, and the outer query then returns only those rows whose CGPA exceeds that computed average.

```
1 SELECT *
2 FROM Student
3 WHERE CGPA > (
4     SELECT AVG(CGPA)
5     FROM Student
6 );
```

Listing 3.10: Query for Task 10



The screenshot shows a SQL query editor interface. The 'Input' tab is active, displaying a SQL query. The query is designed to select all columns from the 'Student' table where the CGPA is greater than the average CGPA of all students. The 'Output' tab shows the results of the query as a table with 7 columns: Student_ID, Student_Name, Gender, Department, Semester, CGPA, and Age. The results list four students: Yousuf (2310023), Mohon (2310025), Moshi (2310027), and Nabil (2310029).

```
SELECT *
FROM Student
WHERE CGPA > (
    SELECT AVG(CGPA)
    FROM Student
);
```

Student_ID	Student_Name	Gender	Department	Semester	CGPA	Age
2310023	Yousuf	Male	EEE	7th	3.85	22
2310025	Mohon	Male	EEE	6th	3.92	22
2310027	Moshi	Male	CSE	7th	3.7	21
2310029	Nabil	Male	BBA	8th	3.95	23

Figure 3.10: Output of Task 10

Chapter 4

Discussion

This lab provided hands-on practice with several fundamental categories of SQL clauses and operators beyond basic data retrieval. Using `UPPER()` and `LOWER()` demonstrated how string functions can transform the format of textual output without changing the underlying stored data. The `AS` keyword showed how column aliasing improves the readability of query results, which is especially useful when working with computed or aggregated columns.

Sorting with `ORDER BY` on multiple columns highlighted how SQL applies a primary sort key followed by a secondary sort key, which is useful for organizing data hierarchically, such as by department and then by performance within that department. Filtering with `BETWEEN`, `IN`, and `NOT IN` illustrated more concise alternatives to chains of comparison and logical operators, making queries easier to read and maintain.

The `GROUP BY` and `HAVING` clauses were particularly important for understanding the difference between filtering individual rows and filtering aggregated groups; while `WHERE` cannot be used with aggregate functions directly, `HAVING` was designed specifically for this purpose. The `UNION` operation demonstrated how two structurally compatible queries can be merged into a single de-duplicated result set, which was clearly visible in Task 9, where names common to both the `Student` and `Alumni` tables appeared only once. Finally, the nested subquery in Task 10 showed how SQL can reference the result of one query inside another, enabling dynamic comparisons, such as against a computed average, without hardcoding a fixed value.

Overall, this lab strengthened the practical understanding of intermediate SQL concepts and demonstrated how these techniques are combined in real-world scenarios to extract meaningful insights from relational data.

Bibliography

- [1] W3Schools, *SQL Tutorial*. [Online]. Available: <https://www.w3schools.com/sql/>
- [2] PostgreSQL Global Development Group, *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/>
- [3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015.
- [4] Department of ECE, RUET, *Database Systems Sessional (ECE 2216) Lab Manual*, 2026.